

# Resolução — Exame Modelo 1 —

## Paradigmas de Programação

---

Época de Recurso | Ano Letivo: 2025/2026

---

### PARTE 1 – Perguntas Teóricas

---

#### Pergunta 1

O encapsulamento é um dos pilares fundamentais da Programação Orientada a Objetos e consiste em ocultar os detalhes internos de implementação de uma classe, expondo apenas uma interface controlada para o exterior. Em Java, este princípio é materializado através dos modificadores de acesso que controlam a visibilidade dos membros de uma classe.

O modificador `private` restringe o acesso ao membro exclusivamente à classe onde foi declarado, impedindo que qualquer classe externa aceda ou modifique diretamente o atributo. O modificador `protected` permite o acesso às subclasses e às classes dentro do mesmo pacote. O modificador `public` permite o acesso universal a partir de qualquer classe em qualquer pacote.

O acesso direto aos atributos é considerado uma má prática porque viola o encapsulamento e impede a validação dos valores atribuídos. Se um atributo for público, qualquer código externo pode atribuir-lhe valores inválidos sem qualquer controlo, comprometendo a integridade do estado do objeto. A utilização de métodos de acesso (getters e setters) permite implementar lógica de validação, garantindo que o objeto mantém sempre um estado consistente e válido.

```
public class Contentor {  
    private String codigo;  
    private double capacidade;  
    private double pesoAtual;  
  
    public Contentor(String codigo, double capacidade) {  
        this.codigo = codigo;  
        this.capacidade = capacidade;  
        this.pesoAtual = 0;  
    }  
  
    public String getCodigo() {  
        return codigo;  
    }  
  
    public double getCapacidade() {  
        return capacidade;  
    }  
  
    public double getPesoAtual() {  
        return pesoAtual;  
    }  
  
    public void setPesoAtual(double peso) {  
        if (peso < 0) {  
            throw new IllegalArgumentException("O peso nao pode ser negativo.");  
        }  
        if (peso > capacidade) {  
            throw new IllegalArgumentException("O peso nao pode exceder a capacidade.");  
        }  
        this.pesoAtual = peso;  
    }  
}
```

---

## Pergunta 2

O polimorfismo é a capacidade de um mesmo elemento assumir múltiplas formas. Em Java, manifesta-se em dois tipos distintos: sobrecarga e sobreposição.

A sobrecarga (overloading) ocorre quando numa mesma classe existem vários métodos com o mesmo nome mas com assinaturas diferentes, ou seja, com diferente número de parâmetros, tipos de parâmetros ou ordem dos tipos. A resolução do método a invocar é efetuada em tempo de compilação pelo compilador, com base nos tipos dos argumentos fornecidos na chamada. A sobrecarga não depende de herança, podendo existir dentro de uma única classe.

A sobreposição (overriding) ocorre quando uma subclasse redefine um método herdado da superclasse, mantendo exatamente a mesma assinatura (nome, número e tipos de parâmetros) e o mesmo tipo de retorno ou um subtipo covariante. A anotação `@Override` é recomendada para garantir que o compilador valida a conformidade da assinatura. A resolução do método a invocar é efetuada em tempo de execução pela JVM, através do mecanismo de ligação dinâmica (dynamic binding), onde a JVM consulta a tabela de métodos virtuais do objeto real na Heap para determinar qual a versão do método a executar.

```
public class Calculadora {  
    public int somar(int a, int b) {  
        return a + b;  
    }  
  
    public double somar(double a, double b) {  
        return a + b;  
    }  
  
    public int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
public class Animal {  
    public void emitirSom() {  
        System.out.println("Som generico.");  
    }  
}  
  
public class Gato extends Animal {  
    @Override  
    public void emitirSom() {  
        System.out.println("Miau!");  
    }  
}  
  
public class Teste {  
    public static void main(String[] args) {  
        Animal a = new Gato();  
        a.emitirSom();  
    }  
}
```

---

### Pergunta 3

A classe Object é a raiz de toda a hierarquia de classes em Java. Todas as classes, quer declarem explicitamente ou não uma superclasse, estendem implicitamente a classe Object. Isto

significa que qualquer objeto em Java herda um conjunto de métodos fundamentais definidos nesta classe.

O método `equals` compara a igualdade lógica entre dois objetos. A implementação por defeito na classe `Object` limita-se a comparar as referências de memória com o operador de igualdade referencial, comportando-se de forma idêntica à comparação de identidade. Deve ser redefinido quando se pretende que dois objetos distintos na memória sejam considerados logicamente iguais com base nos seus atributos. O método `toString` devolve uma representação textual do objeto. A implementação por defeito retorna o nome da classe seguido do símbolo arroba e do código hash em hexadecimal, o que é pouco legível. Deve ser redefinido para fornecer informação descritiva sobre o estado do objeto, facilitando a depuração e a geração de mensagens legíveis. O método `getClass` retorna o objeto `Class` associado à instância, permitindo determinar em tempo de execução qual a classe real do objeto. Não pode ser redefinido porque é declarado como `final`.

Se o método `equals` não for redefinido, dois objetos com o mesmo conteúdo lógico serão considerados diferentes simplesmente porque ocupam posições distintas na memória.

```
public class Teste {  
    public static void main(String[] args) {  
        String s1 = new String("Olá");  
        String s2 = new String("Olá");  
        System.out.println(s1 == s2);  
        System.out.println(s1.equals(s2));  
    }  
}
```

## Pergunta 4

A herança em Java é o mecanismo pelo qual uma classe derivada (subclasse) adquire automaticamente os atributos e métodos de uma classe base (superclasse), permitindo a reutilização e a extensão de funcionalidades existentes. A relação de herança é expressa pela palavra reservada `extends`.

A palavra reservada `super` é utilizada em dois contextos distintos. No contexto de um construtor, a instrução `super` com argumentos invoca o construtor da superclasse, sendo obrigatória quando a superclasse não possui um construtor sem parâmetros. Esta instrução deve ser a primeira no corpo do construtor da subclasse. No contexto de métodos, a instrução `super` seguida do nome do método permite invocar explicitamente a versão da superclasse de um método que foi sobreposto na subclasse.

Java impõe a limitação de herança simples, o que significa que uma classe só pode estender diretamente uma única superclasse. Esta restrição evita problemas de ambiguidade conhecidos como o problema do diamante. Para contornar esta limitação, Java permite que uma classe implemente múltiplas interfaces, herdando assim múltiplos contratos de comportamento sem os problemas da herança múltipla de implementação.

```
public class Veiculo {
    private String matricula;

    public Veiculo(String matricula) {
        this.matricula = matricula;
    }

    public String getMatricula() {
        return matricula;
    }

    public String descricao() {
        return "Veiculo com matricula " + matricula;
    }
}

public class Camiao extends Veiculo {
    private double cargaMaxima;

    public Camiao(String matricula, double cargaMaxima) {
        super(matricula);
        this.cargaMaxima = cargaMaxima;
    }

    @Override
    public String descricao() {
        return super.descricao() + " | Carga maxima: " + cargaMaxima + "kg";
    }
}
```

## PARTE 2 – Programação em Java

---

### Pergunta 1a

```
public class AidBoxImpl implements AidBox {  
    private static final int MAX_CONTAINERS = 4;  
    private String code;  
    private String zone;  
    private Container[] containers;  
    private int numberOfContainers;  
  
    public AidBoxImpl(String code, String zone) {  
        if (code == null) {  
            throw new IllegalArgumentException("O código não pode ser nulo.");  
        }  
        this.code = code;  
        this.zone = zone;  
        this.containers = new Container[MAX_CONTAINERS];  
        this.numberOfContainers = 0;  
    }  
  
    @Override  
    public String getCode() {  
        return this.code;  
    }  
  
    @Override  
    public String getZone() {  
        return this.zone;  
    }  
  
    @Override  
    public Container[] getContainers() {  
        Container[] result = new Container[numberOfContainers];  
        for (int i = 0; i < numberOfContainers; i++) {  
            result[i] = containers[i];  
        }  
        return result;  
    }  
}
```

```
public void addContainer(Container container) throws AidBoxException {
    if (container == null) {
        throw new IllegalArgumentException("O contentor nao pode ser nulo.");
    }
    if (numberOfContainers >= MAX_CONTAINERS) {
        throw new AidBoxException("Capacidade maxima de contentores atingida.");
    }
    for (int i = 0; i < numberOfContainers; i++) {
        if (containers[i].getCode().equals(container.getCode())) {
            throw new AidBoxException("Contentor com este codigo ja existe.");
        }
    }
    containers[numberOfContainers] = container;
    numberOfContainers++;
}

@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (!(obj instanceof AidBox)) {
        return false;
    }
    AidBox other = (AidBox) obj;
    return this.code.equals(other.getCode());
}

@Override
public String toString() {
    return "AidBox [Codigo: " + code + " | Zona: " + zone + " | Contentores: " + numberOfContainers;
}
}
```



## Pergunta 1b

```
public class TestAidBox {  
    public static void main(String[] args) {  
        AidBoxImpl ab1 = new AidBoxImpl("AB-001", "Zona Norte");  
        AidBoxImpl ab2 = new AidBoxImpl("AB-001", "Zona Sul");  
        AidBoxImpl ab3 = new AidBoxImpl("AB-002", "Zona Norte");  
  
        System.out.println("Codigo ab1: " + ab1.getCode());  
        System.out.println("Zona ab1: " + ab1.getZone());  
  
        System.out.println("Contentores iniciais: " + ab1.getContainers().length);  
  
        System.out.println("Igualdade ab1 com ab2 (mesmo codigo): " + ab1.equals(ab2));  
        System.out.println("Igualdade ab1 com ab3 (codigos diferentes): " + ab1.equals(ab3));  
        System.out.println("Igualdade ab1 com nulo: " + ab1.equals(null));  
        System.out.println("Igualdade ab1 com ab1: " + ab1.equals(ab1));  
  
        System.out.println("Representacao textual: " + ab1.toString());  
    }  
}
```

## Pergunta 2a

```
public class ReportImpl implements Report {

    private int countContainersByType(AidBox aidbox, ItemType type) {
        if (aidbox == null || type == null) {
            return 0;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null) {
            return 0;
        }
        int count = 0;
        for (int i = 0; i < containers.length; i++) {
            if (containers[i] != null && containers[i].getType() == type) {
                count++;
            }
        }
        return count;
    }

    private double getAverageOccupancy(AidBox aidbox) {
        if (aidbox == null) {
            return 0;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null || containers.length == 0) {
            return 0;
        }
        double somaOcupacao = 0;
        int contadoresValidos = 0;
        for (int i = 0; i < containers.length; i++) {
            if (containers[i] == null) {
                continue;
            }
            Measurement last = containers[i].getLastMeasurement();
            if (last != null) {
                double ocupacao = (last.getValue() / containers[i].getCapacity()) * 100;
                somaOcupacao += ocupacao;
                contadoresValidos++;
            }
        }
    }
}
```

```
    }  
    if (contadoresValidos == 0) {  
        return 0;  
    }  
    return somaOcupacao / contadoresValidos;  
}  
  
@Override  
public String generate(IInstitution inst) {  
    return "";  
}  
}
```

---

## Pergunta 2b

```
public class ReportImpl implements Report {

    private int countContainersByType(AidBox aidbox, ItemType type) {
        if (aidbox == null || type == null) {
            return 0;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null) {
            return 0;
        }
        int count = 0;
        for (int i = 0; i < containers.length; i++) {
            if (containers[i] != null && containers[i].getType() == type) {
                count++;
            }
        }
        return count;
    }

    private double getAverageOccupancy(AidBox aidbox) {
        if (aidbox == null) {
            return 0;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null || containers.length == 0) {
            return 0;
        }
        double somaOcupacao = 0;
        int contadoresValidos = 0;
        for (int i = 0; i < containers.length; i++) {
            if (containers[i] == null) {
                continue;
            }
            Measurement last = containers[i].getLastMeasurement();
            if (last != null) {
                double ocupacao = (last.getValue() / containers[i].getCapacity()) * 100;
                somaOcupacao += ocupacao;
                contadoresValidos++;
            }
        }
    }
}
```

```
    }

    if (contadoresValidos == 0) {
        return 0;
    }

    return somaOcupacao / contadoresValidos;
}

@Override
public String generate(IIInstitution inst) {
    if (inst == null) {
        return "";
    }

    AidBox[] aidBoxes = inst.getAidBoxes();
    if (aidBoxes == null || aidBoxes.length == 0) {
        return "Sem AidBoxes registradas.";
    }

    String relatorio = "=== RELATORIO DE AIDBOXES ===\n\n";
    int aidBoxesIncluidas = 0;
    for (int i = 0; i < aidBoxes.length; i++) {
        if (aidBoxes[i] == null) {
            continue;
        }

        double mediaOcupacao = getAverageOccupancy(aidBoxes[i]);
        if (mediaOcupacao > 50) {
            relatorio += "AidBox: " + aidBoxes[i].getCode() + "\n";
            relatorio += "Zona: " + aidBoxes[i].getZone() + "\n";
            relatorio += "Ocupacao media: " + mediaOcupacao + "%\n";
            relatorio += "Alim. Pereciveis: " + countContainersByType(aidBoxes[i], ItemType.PERISHABLE) + "\n";
            relatorio += "Alim. Nao Pereciveis: " + countContainersByType(aidBoxes[i], ItemType.NON_PERISHABLE) + "\n";
            relatorio += "Vestuario: " + countContainersByType(aidBoxes[i], ItemType.CLOTHING) + "\n";
            relatorio += "Medicamentos: " + countContainersByType(aidBoxes[i], ItemType.MEDICINE) + "\n";
            relatorio += "---\n";
            aidBoxesIncluidas++;
        }
    }

    if (aidBoxesIncluidas == 0) {
        relatorio += "Nenhuma AidBox com ocupacao superior a 50%.\n";
    }

    return relatorio;
}
```

```
}
```

```
}
```